

MEMORY ALLOCATION SIMULATOR

A Beginner-Friendly Python Implementation

STUDENT INTRODUCTION

Name: Soham R. Dhore
Roll No.: CD25118 | Batch: B3
Course: Python Programming Lab
Course Code: N-PCCCD304P
Branch/Semester: CSE (Data Science) / III Semester

MODULE 1 • INTRODUCTION | MODULE 2 • PROGRAM LOGIC

MODULE 3 • CODE IMPLEMENTATION | MODULE 4 • TESTING & CONCLUSION

MODULE 1

Introduction to the Project

Project idea

This project is a small memory-management simulation. It starts with a total of 1000 KB of sample memory and lets the user allocate part of it to different processes. The program keeps track of the remaining memory after each operation.

WHAT IS MEMORY?	WHAT IS ALLOCATION?	WHAT IS FREEING?
Memory stores data that programs need while they are running.	Giving a process a requested amount of available memory.	Returning a process's allocated memory to the available total.

Project objectives

- Practise Python variables, dictionaries, loops, conditions, and input/output.
- Allocate memory only when the requested size is valid and available.
- Free memory by looking up a process ID.
- Display free memory, used memory, and the active process list.

Basic Python Concepts Used

The program uses basic Python features, suitable for a first programming course.

VARIABLE

Stores a value.

```
memory = 1000
```

DICTIONARY

Stores process IDs and their sizes.

```
used = {}
```

WHILE LOOP

Repeats the menu until the user exits.

IF / ELIF / ELSE

Chooses which action to perform based on the input.

INPUT()

Reads the user's menu choice, process ID, and size.

PRINT()

Displays messages and memory statistics.

Why use a dictionary?

A dictionary links each process ID, such as P1, to the amount of memory assigned to it. This makes it easy to find a process when the user wants to free its memory.

MODULE 1

How the Simulator Works

The program starts with 1000 KB free and no allocated processes. When a process receives memory, the free total decreases. When that process is freed, the amount is added back.

Example memory status

Process	Allocated Size	Status
P1	300 KB	USED
P2	200 KB	USED
Not allocated	500 KB	FREE

FREE = memory still available. USED = memory already assigned to a process.

Important: this is only a learning simulation. It does not change or control the computer's real RAM.

MODULE 1

Problem Statement & Requirements

Problem statement

Create a menu-driven Python program that lets a user allocate memory to a process, free memory for an existing process, view memory statistics, and exit the program.

INPUT	PROCESS	OUTPUT
Menu choice, process ID, and requested memory size.	Check available memory and update the process dictionary.	Show status messages and current memory totals.

Requirements

- Start with 1000 KB of memory.
- Accept only positive allocation sizes that do not exceed free memory.
- Store the allocated size against a process ID.
- Return memory when a known process is freed.
- Display free memory, used memory, and all allocated processes.

Module 1 Summary

Before coding, the main idea is to keep the memory total accurate as processes are added and removed.

- Allocation means assigning part of the available memory to a process.
- Freeing means returning that process's memory to the free total.
- A dictionary stores each process ID and its allocated size.
- A loop repeatedly displays the menu until Exit is selected.
- Conditions validate the choice and decide which operation runs.

Next: Module 2

Module 2 explains the menu workflow and the important Python statements used to implement allocation, freeing, and display operations.

Menu-Driven Program Structure

The menu gives the user four choices. The program reads the choice as text and compares it with values such as "1" or "3".

Choice	Operation	Purpose
1	Allocate	Assign memory to a process.
2	Free	Return memory for a process.
3	Display	Show memory totals and process list.
4	Exit	Stop the loop and end the program.

The while True loop keeps the menu running. The break statement ends it when the user chooses option 4.

MODULE 2

Allocate Memory

The allocation section asks for a process ID and size. It checks that the size is positive and does not exceed the currently available memory.

```
p = input("Process ID: ")
s = int(input("Size: "))

if s > 0 and s <= memory:
    used[p] = s
    memory -= s
    print("Allocated!")
else:
    print("Not enough memory!")
```

Line-by-line explanation

- input() reads the process ID and requested size.
- int() converts the size from text to an integer.
- s > 0 rejects zero or negative requests.
- s <= memory checks that enough free memory remains.
- used[p] = s records the process and its size.
- memory -= s subtracts the allocated amount from free memory.

Free a Process

To free memory, the program checks whether the entered process ID exists in the dictionary.

```
p = input("Process ID: ")

if p in used:
    memory += used.pop(p)
    print("Memory freed!")
else:
    print("Process not found!")
```

Important concepts

- p in used checks whether the process ID is present.
- used.pop(p) removes the process and returns its stored size.
- memory += adds that size back to available memory.
- If the process ID is unknown, no memory is changed and a message is displayed.

Example: if P1 used 300 KB, freeing P1 increases the free-memory total by 300 KB.

Display Memory Details

The display option shows how much memory is free, how much is being used, and the dictionary of active processes.

```
elif ch == "3":
    print("Free memory:", memory, "KB")
    print("Used memory:", 1000 - memory, "KB")
    print("Processes:", used)
```

How the totals work

The initial memory total is 1000 KB. Since memory stores the amount that remains free, used memory can be calculated by subtracting free memory from 1000.

Free Memory	Used Memory	Total
700 KB	300 KB	1000 KB

Algorithm & Flow of Control

Algorithm

- Start the program with memory = 1000 and used = {}.
- Display the menu and ask the user to enter a choice.
- If choice is 1, read process ID and size, validate, then allocate.
- If choice is 2, find and remove the process, then return its memory.
- If choice is 3, display free memory, used memory, and processes.
- If choice is 4, stop the loop.
- Repeat the menu after each operation unless the program exits.

Simple flow

START	MENU	OPERATION	EXIT
Initialize memory and dictionary.	Read the user's choice.	Allocate, free, or display.	Stop when choice 4 is entered.

Complete Python Code — Part 1

This is the same beginner-friendly program provided for implementation. It uses a dictionary to store allocated processes.

```
memory = 1000
used = {}

while True:
    print("\n1.Allocate  2.Free  3.Display  4.Exit")
    ch = input("Choice: ")

    if ch == "1":
        p = input("Process ID: ")
        s = int(input("Size: "))
        if s > 0 and s <= memory:
            used[p] = s
            memory -= s
            print("Allocated!")
        else:
            print("Not enough memory!")
```

In this first part, the program initializes the memory total and empty dictionary, repeats the menu, reads a choice, and handles memory allocation.

Complete Python Code — Part 2

```
    elif ch == "2":
        p = input("Process ID: ")
        if p in used:
            memory += used.pop(p)
            print("Memory freed!")
        else:
            print("Process not found!")

    elif ch == "3":
        print("Free memory:", memory, "KB")
        print("Used memory:", 1000 - memory, "KB")
        print("Processes:", used)

    elif ch == "4":
        break
```

The second part handles freeing memory, displaying statistics, and exiting. Keep the indentation exactly as shown because Python uses indentation to define code blocks.

How to run

Save the code in a file named memory_allocation.py. Run it with Python 3 using the command: python memory_allocation.py

Improvements & Important Notes

The submitted code is kept simple so that a beginner can explain it during a practical or presentation.

INPUT CHECK	MEMORY TRACKING	PROCESS LOOKUP
Rejects zero, negative, and larger-than-available memory sizes.	Updates free memory after allocation and freeing.	Checks whether a process ID exists before freeing it.

Limitations to know

- Entering the same process ID again can replace its old size; use unique IDs such as P1, P2, and P3.
- The program expects the size to be a valid integer.
- This version tracks a total amount of memory; it does not split memory into individual blocks.
- It is a simulation and does not manage actual computer RAM.

Possible future improvement: add duplicate-ID checks, handle invalid numeric input, and implement block management using a linked list.

Testing & Expected Results

Run these test cases during your demonstration to show that each menu option works.

Test	Input / Action	Expected Result
1	Allocate P1, size 300	Allocated; free memory becomes 700 KB.
2	Display memory	Used = 300 KB; free = 700 KB.
3	Free P1	Memory freed; free memory becomes 1000 KB.
4	Free unknown ID P9	Process not found! No memory change.
5	Request size above free memory	Not enough memory!

Sample output

```
Choice: 1
Process ID: P1
Size: 300
Allocated!

Choice: 3
Free memory: 700 KB
Used memory: 300 KB
Processes: {'P1': 300}
```

MODULE 4

Conclusion & Future Scope

Conclusion

This project demonstrates a basic memory allocation simulator using Python variables, a dictionary, loops, conditions, and input/output. It shows how a process can receive memory, how that memory can be returned, and how the current memory status can be displayed.

What I learned

- How to store process information in a dictionary.
- How to repeat a menu using a while loop.
- How to use if/elif conditions to choose an operation.
- How to update totals after allocating and freeing memory.
- How to test normal cases and incorrect requests.

Future scope

The program can be extended by preventing duplicate process IDs, handling invalid input safely, dividing memory into blocks, and using a linked-list data structure to represent memory blocks.

THANK YOU

Soham R. Dhore • CD25118 • Batch B3